

**SIH 26187**

# V.I.E.W

*Virtual Intelligence & Early Warning System*

Border Surveillance & Defense Console

---

## TEAM WORK BREAKDOWN & DOMAIN ASSIGNMENT DOCUMENT

---

Prepared for internal team distribution — Team RootError  
Smart India Hackathon 2026 · Problem Statement 26187 · Border Surveillance System

# Document Contents

---

This document breaks the V.I.E.W project into four working domains, assigns ownership, and defines scope, deliverables, and cross-team dependencies for each. It is meant to be read once as a whole and then used as a standing reference during stand-ups.

1. Purpose & How To Use This Document
2. Team Roster & Domain Map
3. Current Build Status — What Already Exists
4. Domain A — Computer Vision, AI/ML & Model Testing
5. Domain B — Backend, System Architecture, Data & Edge
6. Domain C — Frontend Development & UI/UX Design
7. Domain D — Documentation, QA & Formal Presentation
8. Cross-Cutting Responsibility — Dataset Work (Everyone)
9. Interfaces & Handoff Points Between Domains
10. Communication, Ownership Norms & Meeting Cadence
11. Open Questions To Resolve As A Team

# 1 Purpose & How To Use This Document

This document exists to turn a broad, ten-domain problem breakdown into something a six-person team can actually execute against without stepping on each other. It groups the original domains into four working pillars, assigns clear ownership, and — critically — grounds each pillar in what the current V.I.E.W codebase actually contains today, not just what the problem statement asks for in the abstract.

The original domain breakdown (Computer Vision & ML, Video Processing Pipeline, Backend & System Architecture, Database & Data Management, Frontend & Dashboard, UI/UX & Visual Design, Hardware & Edge Infrastructure, DevOps & Deployment, Testing & Validation, Documentation, and Formal Presentation) is not being discarded. Every one of those eleven areas is still real, still needs doing, and is still referenced in this document — they have simply been folded into four ownership groups so that six people can be meaningfully accountable for all of it without eleven separate conversations.

*Nothing in the original ten-domain list has been dropped. It has been regrouped for ownership, not simplified in scope.*

## How the four domains were formed

- Domain A (Computer Vision & ML Testing) absorbs: Computer Vision & AI/ML, Model Training, and the model-accuracy portion of Testing & Validation.
- Domain B (Backend & System Architecture) absorbs: Backend & System Architecture, Video Processing Pipeline coordination, Database & Data Management, Hardware & Edge Infrastructure, and DevOps/Deployment.
- Domain C (Frontend & UI/UX) absorbs: Frontend & Dashboard and UI/UX & Visual Design in full.
- Domain D (Documentation & Presentation) absorbs: Documentation, Formal Presentation & SIH Deliverables, and acts as the standing QA/proofreading pass across the whole project.

## How to use this in the meeting

- Each domain section below opens with an Owners line and a one-paragraph mandate — read that first so everyone hears their name and their mission in under a minute.
- Sub-sections break the mandate into concrete responsibility clusters. These are scope statements, not a sprint plan — no dates are attached on purpose, so the team can sequence work in a separate planning session once scope is agreed.
- Section 9 (Interfaces & Handoff Points) is the part most likely to cause disagreement — walk through it slowly, since it defines who is blocked on whom.
- Section 11 (Open Questions) is intentionally left for the meeting to resolve live — do not skip it.

## 2 Team Roster & Domain Map

Team RootError consists of six members. The table below is the master assignment — the single source of truth for who owns what. Every later section expands on a row of this table.

|                               |                                                                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Humza Ahmad</b>            | Team Lead · Full-Stack & AI Architect — Domain A (co-owner) + Domain B (co-owner). Cross-domain integration authority.               |
| <b>Suhana Paul</b>            | AI / CV Engineer — Domain A (co-owner). Primary owner of model selection, training, and accuracy validation.                         |
| <b>Mizan Ur Rahman Mondal</b> | Embedded Systems & Hardware Specialist — Domain B (co-owner). Owns hardware/edge subsection and IoT/physical alert integration.      |
| <b>Farhan Ahmad</b>           | Frontend Architect & Integration Lead — Domain C (co-owner). Owns frontend-to-backend wiring and component architecture.             |
| <b>Madhumita Roy</b>          | UI/UX & Graphic Designer — Domain C (co-owner) + Domain D (co-owner). Owns visual design system and shares documentation QA.         |
| <b>Aaratrika Mitra</b>        | Software Associate & Documentation — Domain C (support) + Domain D (co-owner). Owns documentation structure and shares proofreading. |

### Domain-to-people matrix

| Domain                              | Primary Owners        | Supports             | Original Domains Folded In                                      |
|-------------------------------------|-----------------------|----------------------|-----------------------------------------------------------------|
| <b>A — Computer Vision &amp; ML</b> | Suhana, Humza, Farhan | —                    | CV/AI-ML, Model Training, ML-side Testing                       |
| <b>B — Backend &amp; Systems</b>    | Humza, Mizan          | —                    | Backend Architecture, Video Pipeline, DB, Hardware/Edge, DevOps |
| <b>C — Frontend &amp; UI/UX</b>     | Farhan, Madhumita     | Aaratrika            | Frontend & Dashboard, UI/UX & Visual Design                     |
| <b>D — Docs &amp; Presentation</b>  | Madhumita, Aaratrika  | Whole team (reviews) | Documentation, Formal Presentation & SIH Deliverables           |

Note on overlap: Humza sits across Domains A and B by design — as team lead and the person who authored most of the current backend and pipeline scaffolding, he is the natural integration point between AI outputs and system architecture. Madhumita and Aaratrika both appear in two domains because visual design and documentation share a QA instinct — proofreading a dashboard screen and proofreading a report draft use the same eye for polish.

### 3 Current Build Status — What Already Exists

---

Before assigning new work, the team should agree on what is already built, what is scaffolded but incomplete, and what has not been started. This section is a factual snapshot of the repository as it stands today, so nobody duplicates work or assumes something is 'someone else's problem' when it's actually unowned.

#### Fully built and working

- Frontend dashboard (HTML/CSS/JS): the full V.I.E.W console — sensor grid, live feed canvas, alert panel, roster/vehicle/zone modals, system status modal, toasts, and the analog black/white/gold visual language — is complete and functional against demo data.
- Backend REST API surface: Cameras, Alerts, Events, Personnel, Vehicles, and Zones all have full CRUD controllers, Zod validation, pagination, and a consistent error-handling envelope.
- Boot sequence & service resilience: PostgreSQL, Redis, and MQTT all connect with retry/backoff and the server boots gracefully even if any of them are offline — a genuinely solid piece of infrastructure work already in place.
- Demo Mode: an in-memory data store with realistic seeded military data (4 cameras, 5 personnel, 5 vehicles, 4 zones) plus a live WebSocket simulation ticker that fires synthetic human/vehicle/fence events every 3.5 seconds — this is what currently powers the live-feeling dashboard.
- WebSocket layer (Socket.IO): full event contract defined and implemented — demo:init, event:human, event:vehicle, event:fence, alert, plus ping/pong keep-alive.
- Database schema: all 7 PostgreSQL tables (cameras, zones, personnel, vehicles, ai\_events, alerts, users) are defined with UUID keys and proper foreign-key relationships.

#### Scaffolded but incomplete — real work needed

- Authentication: login and refresh endpoints exist and are routed, but password verification (bcrypt) and token refresh logic are explicitly stubbed — the login endpoint currently always throws a 'not implemented' error by design.
- All three AI pipelines (Human Detection, Vehicle Detection, Virtual Fence): the Python pipeline classes, their docstrings, and their intended step-by-step flow are fully written out as comments, but no models are loaded and no inference runs — process() is a placeholder in all three files.
- Camera stream ingestion: the frame demultiplexer that should read RTSP streams and hand frames to the three pipelines in rotation is scaffolded with the right data structures but does not yet read any real stream.
- Zone-to-runtime sync: the backend correctly publishes zone polygon updates to Redis on create/update/delete, but the Python side has no subscriber listening for it yet.
- FCM push notifications: the alert dispatch service has a complete routing structure (HIGH/MEDIUM/LOW/INFO) but the actual Firebase call is commented out.

#### Not started

- Any real model training, dataset collection, or annotation work.
- Any real camera/RTSP hardware integration — everything is currently simulated.
- MinIO/object storage wiring for alert snapshots (config exists, no implementation).

- Role-based permission enforcement on routes (the role hierarchy is documented but not yet enforced beyond a basic guard helper).

*Read literally: the frontend and backend 'shell' are far ahead of the AI layer. This should directly shape how urgently Domain A needs to start, and it's worth saying that out loud in the meeting.*

## 4 Domain A — Computer Vision, AI/ML & Model Testing

**OWNERS:** Suhana Paul (primary) · Humza Ahmad (co-owner / integration)

This is the intelligence core of V.I.E.W — the domain the entire product exists to showcase. Its mandate is to take the three currently-empty pipeline scaffolds (Human Detection, Vehicle Detection, Virtual Fence) and make them real: detecting, classifying, recognising, and tracking against actual video, then validating that the results are accurate enough to trust in a demo and defensible enough to explain to judges.

### 4.1 Human Detection & Troop Verification Pipeline

- Human/person detection using a YOLO-family model (the existing scaffold assumes YOLOv8/v11 — confirm final choice as a team).
- Face detection on the cropped human region (RetinaFace or equivalent).
- Face recognition / embedding generation (ArcFace or FaceNet-class model) to produce a comparable vector per detected face.
- Similarity search against the personnel roster's stored embeddings (FAISS index, per the existing scaffold) to determine verified vs. unidentified.
- Badge/ID OCR as a secondary or fallback verification signal, if pursued.

### 4.2 Vehicle Detection, Classification & ANPR

- Vehicle detection and bounding-box extraction from frames.
- Military vehicle type classification (Combat / Transport / Patrol, matching the existing schema).
- Number-plate region detection, perspective correction, and upscaling — all three steps are already sketched in the pipeline comments.
- OCR processing on the corrected plate crop (PaddleOCR is the current assumption; EasyOCR is listed as an alternative in requirements.txt).
- Cross-referencing recognised plates against the vehicle registry to flag unregistered/civilian vehicles.

### 4.3 Virtual Fence & Intrusion Detection

- Combined human + vehicle detection feeding a shared tracker (ByteTrack, per the existing scaffold).
- Object tracking across frames so the same entity keeps one ID as it moves.
- Point-in-polygon zone testing per tracked centroid (the Shapely-based logic already exists structurally in virtual\_fence.py — it needs a real tracker and real detections feeding it).
- Dwell-timer logic per zone type (RED zones alert instantly; AMBER/CORRIDOR zones alert after a threshold).
- Rule-based behaviour/anomaly flags on top of raw detection, e.g. loitering, wrong-direction movement through a corridor zone.

### 4.4 Model Training & Lifecycle

- Dataset collection, cleaning, and annotation — see Section 8, this is a whole-team responsibility, but Domain A defines the annotation spec (bounding box format, class list, face-recognition sample requirements) that everyone else annotates against.
- Model selection and justification (why YOLOv8/v11 over alternatives; why ArcFace over FaceNet, etc.) — this doubles as content Domain D will need for the presentation.
- Fine-tuning pretrained models on the collected/annotated dataset rather than training from scratch.
- Training runs, validation splits, and iteration.

#### 4.5 Testing, Accuracy & Performance Validation

- Detection accuracy testing (precision/recall) per pipeline.
- False-positive rate testing — especially important for the Virtual Fence pipeline, since false HIGH alerts directly undermine operator trust in a live demo.
- OCR accuracy testing on plates under varied angles/lighting.
- FPS and inference-latency benchmarking per pipeline, and combined throughput when all three run concurrently on the alternating-frame schedule.
- GPU/VRAM benchmarking to inform the hardware requirements Domain B needs to document.

#### What Domain A hands off to Domain B

- A defined Redis payload contract per pipeline (the minimum shape — camera\_id, frame\_index, timestamp\_utc, confidence — already exists; Domain A needs to finalise the full per-pipeline payload fields such as entity\_name, vehicle\_type, zone\_id).
- The trained model files/weights, packaged in a way Domain B can load into the FastAPI runtime without needing to know how they were trained.
- Confirmed hardware/runtime requirements (GPU memory, expected FPS) so Mizan can plan the edge/hardware story accurately.

## 5 Domain B — Backend, System Architecture, Data & Edge

**OWNERS:** Humza Ahmad (primary / architecture) · Mizan Ur Rahman Mondal (co-owner / hardware & edge)

This domain is the connective tissue of the whole system: the Node.js/Hono middleware, the PostgreSQL/Redis/MQTT data layer, the Python FastAPI runtime shell that will eventually host Domain A's models, and — where relevant for a hackathon prototype — the physical/edge hardware story. Much of the 'plumbing' already exists and works; this domain's job is largely completion, hardening, and connecting real data where demo data currently stands in.

### 5.1 Core Backend API & Service Architecture

- Complete authentication: implement bcrypt password comparison in authController.ts (currently intentionally stubbed) and implement the token refresh endpoint.
- Enforce role-based permissions across protected routes per the documented role matrix (admin / operator / viewer) — the requireRole() guard exists but is not yet applied where the docs say it should be.
- Maintain and extend the REST surface for Cameras, Alerts, Events, Personnel, Vehicles, and Zones as new fields or filters are needed by Domain A or Domain C.
- Own the global error-handling contract, request validation (Zod), and the consistent JSON error envelope already established — keep new endpoints consistent with it.

### 5.2 Video Processing & Pipeline Coordination

- Build the real RTSP/IP camera stream reader in camera\_stream.py — currently a placeholder that never opens a real stream.
- Implement the frame demultiplexer that assigns frames round-robin to the Human / Vehicle / Virtual Fence pipelines (the slot logic 0/1/2 already exists structurally).
- Wire the three pipeline classes from Domain A into main.py's lifespan — currently commented out — so they actually initialise and receive frames.
- Own the FastAPI internal service (port 8000): health and pipeline-status endpoints already exist as placeholders and need to reflect real pipeline state.

### 5.3 Real-Time Event & Alert Pipeline

- Ensure Redis Pub/Sub channels (ai:human, ai:vehicle, ai:fence, ai:alert) carry Domain A's finalised payload shape end-to-end into redisSubscriber.ts and then into ai\_events.
- Complete the TODO'd DB-write handlers referenced in redisSubscriber.ts (handleHumanEvent, handleVehicleEvent, handleFenceEvent already exist in eventsController.ts and need to be actually called from the subscriber, not just logged).
- Implement the config:zones subscriber on the Python side so zone edits made in the dashboard actually reach the Virtual Fence pipeline live, as the backend already promises.
- Finish FCM push integration (currently a logged placeholder in alerts.ts) or make a deliberate team call to drop it from MVP scope.

## 5.4 Database & Data Management

- Own the 7-table PostgreSQL schema (cameras, zones, personnel, vehicles, ai\_events, alerts, users) — schema.sql already reflects the target design.
- Own the personnel and vehicle registries as the systems of record that Domain A's recognition/ANPR results are checked against.
- Own the FAISS vector index for face embeddings — coordinate directly with Suhana on embedding format and refresh strategy when personnel are added/removed.
- Own audit-quality event logging — every AI detection, its confidence, its zone, and its resulting alert should be reconstructable from the DB after the fact.
- Plan and own the eventual migration off in-memory Demo Mode to real persisted data — a deliberate cutover, not an accident.

## 5.5 Hardware & Edge Infrastructure

*This subsection is intentionally lightweight for a software-first SIH prototype — the goal is a credible story and a working demo of the concept, not a hardware bring-up project.*

- Source/confirm the camera or video source used for the live demo (real webcam/RTSP feed vs. pre-recorded footage looped as a virtual stream).
- MQTT edge-signal integration: the client already exists and publishes to alerts/<camera\_id> — decide on and build a simple physical demonstration (e.g. an LED, buzzer, or even a browser-based sound as a stand-in) so the 'edge alert' concept is tangible for judges.
- Document realistic GPU/edge hardware requirements (Jetson-class device vs. server GPU) for a production deployment, informed by Domain A's benchmarking — this feeds directly into Domain D's architecture slides.
- Network/connectivity considerations for a real border deployment — this can stay at the research/documentation level for the prototype.

## 5.6 DevOps, Environment & Deployment

- Own environment configuration (.env, config/index.ts) and keep it in sync as new variables are needed.
- Own the Python virtual environment and CUDA/GPU setup instructions for whichever machine will run the live demo.
- Keep the repository's build/dev scripts (pnpm dev, uvicorn) working and documented as the project grows.
- Own performance testing of the integrated system once Domain A's pipelines are wired in — confirm the whole stack holds up under the demo's expected load, not just each piece in isolation.

## 6 Domain C — Frontend Development & UI/UX Design

**OWNERS:** Farhan Ahmad (primary / integration) · Madhumita Roy (primary / visual design) · Aaratrika Mitra (support)

Unlike Domain A, this domain starts from a position of strength: the V.I.E.W dashboard is already a complete, polished, single-page application with its own renderer, HUD canvas animations, and a fully defined visual identity. The job here is less 'build the frontend' and more 'connect it to reality, extend it responsibly, and keep the design language consistent as real data and real roles get introduced.'

### 6.1 Frontend Architecture & Real-Data Integration

- Transition the dashboard from Demo Mode (`api.summary()`, the `FALLBACK` object in `main.js`) to authenticated, real API calls once Domain B's auth and real data are ready — this is the single biggest piece of net-new frontend work.
- Implement the login flow against `/api/auth/login` once Domain B completes it — currently no login UI exists because the backend endpoint isn't functional yet.
- Extend the existing camera, alert, roster, vehicle, and zone modals to support create/edit actions (currently mostly read-only views over demo data), matching the CRUD endpoints Domain B already exposes.
- Build a zone-drawing interface for the Zones modal — letting an operator define a polygon visually rather than only via raw coordinates, feeding directly into the `POST /api/zones` contract.
- Maintain the `Socket.IO` event handling in `api.js` and `main.js` as new real-time event fields are finalised by Domain A/B.

### 6.2 Real-Time & Live Feed Experience

- Own the HUD canvas rendering system (the procedural `Scene` class in `ui.js`) — decide, with Domain A/B, how much of this stays as a stylised radar/HUD overlay versus needing to render actual video frames with real bounding boxes.
- If real video is introduced, own the transition from the current canvas-only simulation to an actual video element with a bounding-box overlay layer synced to AI event data.
- Own alert-feed UX: severity colour coding, audio alert tones (already implemented via `WebAudio`), and the `acknowledge/dismiss/escalate` flows already scaffolded in the alert modal.

### 6.3 UI/UX & Visual Design System

- Own and evolve the 'Dieter Rams' analog design language — zero border-radius, strict black/white/gold palette, JetBrains Mono for telemetry — as new screens are added, so nothing drifts off-brand.
- Design any new screens needed as the product grows (e.g. a login screen, a zone-drawing tool, a personnel-add form) in a style consistent with the existing dashboard.
- Own iconography and any additional graphics needed for the live demo or presentation deck (handoff point with Domain D).
- Review information hierarchy on the alert feed and stats panel as real data volume (vs. the current capped 20-alert demo list) is introduced — decide if pagination, filtering, or search is needed.

## 6.4 Responsiveness, Accessibility & Polish

- Test and refine the existing responsive breakpoints (the CSS already includes tablet/narrow-viewport rules) against whatever device the live demo will actually run on.
- General QA pass on interactive states (hover, active, focus) across buttons, modals, and list items — this overlaps with Domain D's broader QA pass but the frontend-specific polish sits here.

## What Domain C depends on from other domains

- From Domain B: working authentication, and real (non-demo) responses from every CRUD endpoint it currently only reads from demoStore.
- From Domain A (via Domain B): the finalised shape of event:human / event:vehicle / event:fence payloads, since the frontend's rendering logic (onHuman, onVehicle, onFence in main.js) currently expects the demo simulation's field names specifically.

## 7 Domain D — Documentation, QA & Formal Presentation

**OWNERS:** Madhumita Roy (co-owner) · Aaratrika Mitra (primary)

This domain is not an afterthought bolted on at the end — it is the domain that determines whether judges and outside developers can actually understand what the other three domains built. It also serves as the project's standing quality gate: catching logic errors, inconsistent wording, and gaps between what the code does and what the team says it does.

### 7.1 Technical Documentation

- Own and keep current the master technical reference (the project already has a strong 15-section `BACKEND_DOCUMENTATION.md` — extend the same rigor to the AI runtime and frontend once those mature).
- Document system architecture, including the boot sequence, service dependency graph, and the three-pipeline frame-alternation design — translating Domain B's implementation into diagrams and plain language.
- Document the database schema and the reasoning behind key design choices (e.g. why `alerts.id` mirrors `ai_events.id`, why zones publish to Redis on every mutation).
- Document each AI model used, why it was chosen, and its measured accuracy/performance — sourced directly from Domain A's validation work in Section 4.5, not invented independently.
- Maintain the installation guide and README as environment requirements evolve (the current README is already thorough — keep it synced with reality as auth, RTSP, and DevOps details firm up).

### 7.2 Logic, Wording & Cross-Document QA

- Proofread all technical documentation for internal consistency — e.g. confirming that documented behaviour (such as the role-permission table) actually matches what Domain B has implemented at any given time, and flagging the gap if not.
- Review naming consistency across the codebase's documentation surface (API field names, event names, status enums) so the docs never silently drift from the code.
- Act as a standing second pair of eyes on any team-facing written material — this document included — for clarity, tone, and correctness before it's shared externally or with judges.
- Flag logical gaps between domains early — e.g. if Domain C builds a UI for a field Domain B's API doesn't return, or if Domain A's payload doesn't match what Domain B's DB schema expects.

### 7.3 Formal Presentation & SIH Deliverables

- Own the pitch deck (PPT) structure: problem framing, solution overview, architecture diagrams, workflow diagrams, and technology-stack justification.
- Own the demo video/live-demo narrative — sequencing which features to show and in what order to make the strongest case in the available time.
- Own UI screenshots and any presentation-specific graphics, coordinating directly with Madhumita/Domain C for source assets.
- Prepare innovation-highlight and future-scalability talking points, and draft anticipated Q&A responses — informed by real technical detail from Domains A and B rather than generic claims.

## What Domain D depends on from other domains

- From Domain A: final model choices, accuracy numbers, and dataset details, to write both the technical documentation and the presentation honestly.
- From Domain B: an accurate, current architecture to diagram — this domain should always be documenting the real system, not the aspirational one.
- From Domain C: final screenshots and a stable UI to photograph/record once the frontend has moved past demo-data mode.

## 8 Cross-Cutting Responsibility — Dataset Work (Everyone)

**OWNERS:** All six members — no exceptions

Dataset collection and annotation is explicitly called out as a shared responsibility, not something that belongs to Domain A alone. Model quality is bottlenecked on data volume and label quality far more than on model architecture choice, and six people annotating in parallel is dramatically faster than one or two.

### What 'everyone' means in practice

- Suhana and Humza (Domain A) define the annotation specification: class list, bounding-box format, minimum samples per class, and any face-recognition-specific sampling requirements (angles, lighting, occlusion variety).
- Once that spec exists, all six members — regardless of domain — take assigned batches of raw footage/images and annotate them to spec. This is treated as a shared team task with its own short check-in, not folded silently into any one domain's workload.
- Mizan, given the hardware/edge role, is well placed to help source or capture realistic footage (camera angles, distances, lighting conditions) that will actually resemble the deployment scenario, rather than relying solely on stock datasets.
- Domain D should track dataset provenance (sources, licensing, size, class balance) as it's collected, since this becomes required documentation content later — collecting this information at annotation time is far easier than reconstructing it afterward.

### Suggested split of the annotation workload

The exact split should be decided in the meeting once the annotation spec and total volume are known, but a reasonable default is to divide by pipeline rather than by person count — e.g. two members focus primarily on human/face data, two on vehicle/plate data, and two on virtual-fence/tracking scenarios — while everyone remains available to help wherever the backlog is largest closer to the demo date.

## 9 Interfaces & Handoff Points Between Domains

Most cross-team friction on a project like this comes from unclear handoffs, not unclear ownership. This section makes explicit exactly what crosses each domain boundary, in which direction, and in what shape.

| From → To | Interface              | What crosses the boundary                                                                                                                             |
|-----------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| A → B     | Redis Pub/Sub channels | ai:human / ai:vehicle / ai:fence / ai:alert JSON payloads — final field-by-field shape must be agreed before Domain B finishes its DB-write handlers. |
| A → B     | Model artifacts        | Trained weights + a documented loading procedure the FastAPI runtime can call without needing training context.                                       |
| B → A     | config:zones channel   | Live zone-polygon updates the Virtual Fence pipeline must subscribe to and hot-reload.                                                                |
| B → C     | REST API + Socket.IO   | Real (non-demo) responses from every CRUD endpoint; a working /api/auth/login; finalised WebSocket event field names.                                 |
| C → B     | Zone-drawing payloads  | Polygon coordinate arrays from the new zone-drawing UI, matching the existing POST /api/zones contract exactly.                                       |
| A, B → D  | Technical facts        | Model choices, accuracy figures, architecture decisions, and their reasoning — Domain D documents reality, it does not invent it.                     |
| C → D     | Visual assets          | Screenshots, screen recordings, and design assets once the UI is stable enough to represent the final product.                                        |
| D → All   | QA feedback            | Flags on inconsistencies between what's documented/presented and what's actually implemented, routed back to the owning domain.                       |

## 10 Communication, Ownership Norms & Meeting Cadence

---

This section is deliberately light — the goal is a small set of norms the team can actually sustain, not a heavyweight process.

### Ownership norms

- Each domain's primary owner(s) have final say on scope decisions within that domain, but any change that crosses a handoff point in Section 9 should be flagged to the other side before it's finalised, not after.
- If a task doesn't clearly belong to any one domain, it defaults to whichever domain is closest to the codebase it touches, with Humza as the tiebreaker given the team-lead / cross-domain role.
- Dataset annotation (Section 8) is the one task type that is explicitly nobody's sole responsibility — it should never be assumed to be 'Domain A's problem' by default.

### Suggested cadence

- A short cross-domain sync (even 10–15 minutes) makes sense whenever a Section 9 handoff is about to change shape — e.g. right before Domain A finalises its event payload format, or right before Domain B ships working auth.
- Domain D should sit in on at least the tail end of Domain A and Domain B discussions periodically, simply to keep documentation from falling out of sync — it's far cheaper to document as you go than to reconstruct decisions after the fact.

### Decision log

It's worth deciding today whether the team keeps a lightweight shared decision log (even a single shared doc) for things like 'we chose YOLOv8 over v11 because X' or 'we're dropping FCM push from MVP scope.' This document assumes such decisions exist somewhere Domain D can reference them.

## 11 Open Questions To Resolve As A Team

---

These are the questions this document surfaced while mapping the codebase to the domain breakdown. They're worth resolving explicitly in today's meeting rather than letting each domain assume an answer independently.

### For Domain A

- Final model choices: YOLOv8 vs YOLOv11 for detection; ArcFace vs FaceNet/DeepFace for face recognition (the README references YOLOv11/FaceNet-DeepFace; the actual pipeline code comments reference YOLOv8/ArcFace/FAISS — these two references currently disagree and should be reconciled).
- Target dataset size and annotation deadline, so Section 8's shared workload can be scoped concretely.

### For Domain B

- Whether FCM push notifications are in scope for the SIH demo at all, or explicitly deferred — this affects both backend work and what Domain D should claim in the presentation.
- Whether the live demo will use a real camera feed, pre-recorded looped footage, or stays on simulated Demo Mode data for the actual judging — this materially changes Domain B's and Mizan's workload.

### For Domain C

- Whether the HUD canvas (currently a stylised procedural animation) should be replaced with real video + bounding-box overlays for the final demo, or kept as-is for visual polish — this is really a joint decision with Domain A/B depending on the camera-feed answer above.

### For Domain D

- Confirm the actual submission format and page/slide limits for the current SIH 26187 cycle, since deliverable structure should be built against real constraints rather than assumed ones.

### For the whole team

- Confirm today's meeting produces at least: a finalised dataset annotation spec, an agreed model shortlist, and an agreed answer on the real-camera-vs-simulated-demo question — these three unlock the most work across every other domain.

---

*End of document. This is a living reference — update the ownership table in Section 2 if roles shift, and keep Section 3's build-status snapshot current as stubbed features get completed.*